



Universidade do Porto
Faculdade de Engenharia

FEUP

Faculdade de Engenharia da Universidade do Porto
Licenciatura em Engenharia Informática e Computação

Programação em Lógica

2002/2003
LEIC

Exame de 11.01.2003 - Resolução

% Grupo 1 - Operações com Listas

% 1) remove_todos(+X, +Lista, ?Nova).

remove_todos(_, [], []) :- !.

remove_todos(X, [X|T], T2) :- !, **remove_todos**(X, T, T2).

remove_todos(X, [H|T], [H|T2]) :- **remove_todos**(X, T, T2).

% 2) extremos(+Lista, -Maximo, -Minimo).

extremos([H|T], Max, Min) :- **extremos2**(T, H, H, Max, Min).

extremos2([], Max, Min, Max, Min) :- !.

extremos2([H|T], Max, Min, MaxFim, MinFim) :-

(H > Max, Max2 = H ; Max2 = Max),

(H < Min, Min2 = H ; Min2 = Min), !,

extremos2(T, Max2, Min2, MaxFim, MinFim).

% 2) extremosb(+Lista, -Maximo, -Minimo). //Solução mais compacta

extremosb([H|[]], H, H).

extremosb([H|T], Max, Min) :-

extremosb(T, Max2, Min2),

(H > Max2, Max = H ; Max = Max2),

(H < Min2, Min = H ; Min = Min2), !.

% 3) Dados do Problema

europa(lisboa, portugal, capital, 900).

europa(porto, portugal, cidade, 300).

europa(madrid, espanha, capital, 3100).

europa(barcelona, espanha, cidade, 2900).

europa(espinho, portugal, cidade, 40).

europa(gaia, portugal, cidade, 260).

europa(arcozelo, portugal, vila, 15).

europa(coimbra, portugal, cidade, 105).

```
% 3) a) cidades_grandes(+Pais, -Lista, -Pop_Total)
```

```
cidades_grandes(Pais, Lista, Pop_Total):-  
    findall( (Cid, Pop),  
        (europa(Cid, Pais, cidade, Pop), Pop > 100),  
        Lista),  
    soma_lista(Lista, 0, Pop_Total).
```

```
soma_lista([], SF, SF).  
soma_lista([_,P]|T], S, SF):- S2 is S+P, soma_lista(T, S2, SF),!.
```

```
% 3) b) capitais(-Lista)
```

```
capitais(Lista):-  
    findall( Cap*Num,  
        (europa(Cap, Pais, capital, _), conta_cidades(Pais, Num)),  
        Lista).
```

```
conta_cidades(Pais, Num):-  
    findall( Cid,  
        (europa(Cid, Pais, Tipo, _), Tipo\==vila),  
        Lista),  
    conta_lista(Lista,0,Num). % length(Lista, Num)
```

```
conta_lista([],CF,CF).  
conta_lista([_|T], C, CF):- C2 is C+1, conta_lista(T, C2, CF),!.
```

```
% Grupo 2 - Jogo do Solitario
```

```
% Predicados auxiliares
```

```
member(X, [X|_]) :- !  
member(X, [_|Resto]) :- member(X, Resto).
```

```
append([], L, L).  
append([X|L1], L2, [X|L3]) :- append(L1, L2, L3).
```

```
tabuleiro( [(1,3,o),(1,4,o),(1,5,o),(2,3,o),(2,4,o),(2,5,o),  
    (3,1,o),(3,2,o),(3,3,o),(3,4,o),(3,5,o),(3,6,o),(3,7,o),  
    (4,1,o),(4,2,o),(4,3,o),(4,4,x),(4,5,o),(4,6,o),(4,7,o),  
    (5,1,o),(5,2,o),(5,3,o),(5,4,o),(5,5,o),(5,6,o),(5,7,o),  
    (6,3,o),(6,4,o),(6,5,o),(7,3,o),(7,4,o),(7,5,o)].
```

```
tabuleiro2( [(1,3,o),(1,4,o),(1,5,o),(2,3,o),(2,4,o),(2,5,o), % teste!  
    (3,1,x),(3,2,x),(3,3,o),(3,4,o),(3,5,o),(3,6,x),(3,7,x),  
    (4,1,x),(4,2,x),(4,3,x),(4,4,o),(4,5,x),(4,6,x),(4,7,x),  
    (5,1,x),(5,2,x),(5,3,x),(5,4,x),(5,5,x),(5,6,x),(5,7,x),  
    (6,3,x),(6,4,x),(6,5,x),(7,3,x),(7,4,x),(7,5,x)].
```

```
teste:- tabuleiro2(B), mostra_tab(B), resolve_solitario(B, Lista),
nl, write(Lista).
```

```
% 1) mostra_tab(+Board)
%
%      1 2 3 4 5 6 7
%  #  #  #  #  #  #  #  #
%1 #  #  #  o  o  o  #  #  #  1
%2 #  #  #  o  o  o  #  #  #  2
%3 #  o  o  o  o  o  o  #  3
%4 #  o  o  o  .  o  o  o  #  4
%5 #  o  o  o  o  o  o  o  #  5
%6 #  #  #  o  o  o  #  #  #  6
%7 #  #  #  o  o  o  #  #  #  7
%  #  #  #  #  #  #  #  #
%      1 2 3 4 5 6 7
```

```
mostra_tab(Board):-
    nl, linha_num, linha_fora,
    mostra_linhas(1,Board),
    linha_fora, linha_num, nl, !.
linha_num:- write('      1 2 3 4 5 6 7'), nl.
linha_fora:- write('  #  #  #  #  #  #  #  #'), nl.
```

```
mostra_linhas(8, _).
mostra_linhas(Lin,Board):-
    write(Lin), write(' # '),
    mostra_linha(Lin, 1, Board),
    write('# '), write(Lin), nl,
    Lin2 is Lin + 1,
    mostra_linhas(Lin2, Board).
```

```
mostra_linha(_, 8, _).
mostra_linha(Lin, Col, Board):-
    (member((Lin,Col,Val), Board), escreve(Val) ; escreve(v)),
    Col2 is Col + 1,
    mostra_linha(Lin, Col2, Board).
```

```
escreve(x):- write('. ').
escreve(o):- write('o ').
escreve(v):- write('# ').
```

```
% 2) Vizinho(+Board, ?Xi, ?Yi, ?Xf, ?Yf, ?Direc).
```

```
vizinho(Board, Xi, Yi, Xf, Yf, Dir):-  
    direccao(Xi, Yi, Xf, Yf, Dir),  
    member((Yi, Xi, _), Board),  
    member((Yf, Xf, _), Board).
```

```
direccao(Xi, Y, Xf, Y, dir):- nonvar(Xi), Xf is Xi + 1.  
direccao(X, Yi, X, Yf, cim):- nonvar(Yi), Yf is Yi + 1.  
direccao(Xi, Y, Xf, Y, esq):- nonvar(Xi), Xf is Xi - 1.  
direccao(X, Yi, X, Yf, bai):- nonvar(Yi), Yf is Yi - 1.
```

```
direccao(Xi, Y, Xf, Y, dir):- nonvar(Xf), Xi is Xf - 1.  
direccao(X, Yi, X, Yf, cim):- nonvar(Yf), Yi is Yf - 1.  
direccao(Xi, Y, Xf, Y, esq):- nonvar(Xf), Xi is Xf + 1.  
direccao(X, Yi, X, Yf, bai):- nonvar(Yf), Yi is Yf + 1.
```

```
% 3) resolve_solitario( +Board, -Lista_Jogadas)
```

```
resolve_solitario(Board, []):-  
    testa_fim(Board), nl, write('Fim!'), nl, mostra_tab(Board), nl.  
resolve_solitario(Board, [(Xi/Yi-Xf/Yf)|Resto]):-  
    jogada_valida(Board, Xi/Yi, Xf/Yf),  
    executa_jogada(Board, Xi/Yi, Xf/Yf, NewBoard),  
    resolve_solitario(NewBoard, Resto).
```

```
testa_fim(Board):- member((Y,X,o), Board), (X\=4 ; Y\=4), !, fail.  
testa_fim(_).
```

```
% Predicados auxiliares:
```

```
jogada_valida(Board, Xi/Yi, Xf/Yf):-  
    member((Yi,Xi,o), Board),  
    vizinho(Board, Xi, Yi, Xm, Ym, Dir),  
    member((Ym,Xm,o), Board),  
    vizinho(Board, Xm, Ym, Xf, Yf, Dir),  
    member((Yf,Xf,x), Board).
```

```
executa_jogada(Board, Xi/Yi, Xf/Yf, NewBoard):-  
    vizinho(Board, Xi, Yi, Xm, Ym, Dir),  
    vizinho(Board, Xm, Ym, Xf, Yf, Dir),!,  
    append(La1, [(Yi,Xi,o)|Lb1],Board),  
    append(La1, [(Yi,Xi,x)|Lb1],Board2),!,  
    append(La2, [(Ym,Xm,o)|Lb2],Board2),  
    append(La2, [(Ym,Xm,x)|Lb2],Board3),!,  
    append(La3, [(Yf,Xf,x)|Lb3],Board3),  
    append(La3, [(Yf,Xf,o)|Lb3],NewBoard),!.
```